

Wednesday 5:46 PM



lab_one.pdf
PDF

Here is my lab, run me though what we doing, what language, what is the task, how can i go about it

Worked for 1m 7s

Ari, this lab is asking you to build a **small full-stack application**, not merely design a webpage.

1. What are you building?

A **local todo/task manager** that:

- Runs on the user's computer using Node.js.
- Opens in a browser through Next.js.
- Stores information permanently in a local SQLite database.
- Has no login system and supports one local user.

The required functionality is:

- Create tasks.
- Edit tasks.
- Archive tasks — **never permanently delete them**.
- View active and archived tasks.
- Sort tasks by:
 - Topic
 - Status
 - Due date
- Give each task exactly one of these statuses:
 - Todo
 - In-Progress
 - Complete
- Visibly flag overdue tasks without making `Overdue` a status.
- Preserve everything after the application is stopped and restarted.

lab_one

2. What language are you using?

Next.js is not a programming language. It is a web-development framework.

I recommend using:

- **TypeScript** — your main programming language.
- **React/TSX** — for forms, buttons and task displays.
- **SQL** — directly or indirectly for SQLite.
- **CSS** — for appearance.
- **Node.js and npm** — to install and run the project.

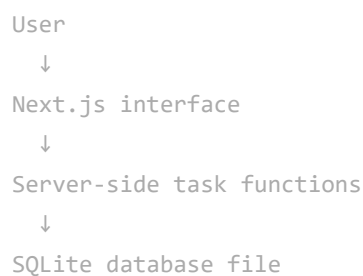
Your files will look roughly like:

app/page.tsx	Main task page
app/actions.ts	Create/edit/archive operations
components/TaskForm.tsx	Task form
components/TaskList.tsx	Task list
lib/database.ts	Database connection
prisma/schema.prisma	Database structure
tests/tasks.test.ts	Automated tests
README.md	Running instructions

The exact structure can vary.

3. How the application works

Think of it as four layers:



For example:

1. The user completes the "Create Task" form.
2. Next.js receives the submitted values.
3. Your server-side code validates them.
4. The task is inserted into SQLite.
5. The page reloads or refreshes the task list.
6. Because it is stored in SQLite, it still exists after restarting the app.

SQLite will create a local database file such as:

```
dev.db
```

That database file is the reason the tasks persist.

4. Your main database table

You probably only need one main table:

```
Task
-----
id
title
description
dueDate
topic
status
archivedAt
createdAt
updatedAt
```

A sensible design would be:

id	Integer, primary key
title	Text, required
description	Text
dueDate	Date/time, required
topic	Text, required
status	Todo In-Progress Complete
archivedAt	Date/time or null
createdAt	Date/time
updatedAt	Date/time

Archiving

Do not delete the row and do not move it into another table.

When archived:

```
archivedAt = current date and time
```

When active:

```
archivedAt = null
```

Then:

- Active page: show tasks where `archivedAt` is null.
- Archived page: show tasks where `archivedAt` is not null.

Overdue tasks

Do **not** store this:

```
isOverdue = true
```

Calculate it whenever tasks are displayed:

```
const overdue =  
  task.dueDate < new Date() &&  
  task.status !== "Complete";
```

The marking rubric specifically warns against storing overdue as a database column or status. It must be derived from the due date and status.

lab_one

5. Recommended technology choice

A straightforward stack would be:

```
Next.js  
TypeScript  
SQLite  
Prisma  
Vitest
```

Why:

- **Next.js**: required by the brief.
- **TypeScript**: catches mistakes before runtime.
- **SQLite**: required by the brief.
- **Prisma**: provides a readable schema and database migrations.
- **Vitest**: runs automated tests with one command.

Do not install fifteen UI libraries because a YouTuber had a spiritual experience with gradients. The functionality carries most of this lab.

6. Starting the project

One workable starting route is:

```
npx create-next-app@latest todo-lab --typescript --eslint --app
cd todo-lab
npm install prisma @prisma/client
npx prisma init --datasource-provider sqlite
```

That gives you:

- A Next.js project.
- TypeScript.
- A Prisma database configuration.
- SQLite as the database provider.

You would then define your `Task` model in:

```
prisma/schema.prisma
```

After defining the model:

Bash

This creates the database and migration files.

Commit immediately after the basic project starts:

```
git add .
git commit -m "Set up Next.js project with TypeScript"
git push
```

7. Build it in this order

Stage 1: Get an empty application running

Make sure:

```
npm install
npm run dev
```

starts the application.

Do not begin styling yet. First prove that the project works.

Stage 2: Build the database

Create the Task model with:

- Title
- Description
- Due date
- Topic
- Status
- Archive timestamp

Add constraints so that status cannot become something ridiculous like `Mostly Done` Perhaps .

Stage 3: Create tasks

Build a form containing all four required task fields:

```
Title
Description
Due Date
Topic
```

Status may default to `Todo` , although allowing it to be selected is also reasonable.

After submission, insert the task into SQLite and show it in the list.

Stage 4: Display tasks

Each task should display:

```
Title
Description
Topic
Due date
Status
Edit button
Archive button
```

Stage 5: Edit tasks

The user must be able to change an existing task.

The edited value must survive:

- A browser reload.
- Stopping and restarting the application.

That means the edit must update SQLite, not merely React state.

Stage 6: Archive tasks

Clicking Archive should:

```
Set archivedAt
Remove task from active list
Keep task visible under Archived
```

There must be no normal Delete button.

Stage 7: Add sorting

Use a dropdown such as:

```
Sort by:
- Due date
- Topic
- Status
```

Sorting can happen:

- In the SQL query, or
- In TypeScript after retrieving the tasks.

Database sorting is cleaner, but either can satisfy the visible functionality.

Stage 8: Flag overdue tasks

An overdue task could have:

- A red border.
- An "Overdue" badge.
- Red due-date text.
- A warning icon.

It must remain one of the three normal statuses.

Stage 9: Add tests

You need at least three meaningful tests.

Good choices:

1. Creating a task stores all its fields.
2. Archiving removes a task from the active query but keeps it in the database.

3. A past-due incomplete task is recognised as overdue.
4. Completed tasks are not flagged as overdue.
5. Sorting by due date returns tasks in the correct order.

The tests must use a disposable test database rather than depending on your personal `dev.db`.

Your README should give one test command, such as:

```
Bash
```

The rubric requires at least one test involving archiving or overdue behaviour.

lab_one

8. Documentation you must submit

Your markdown documentation needs these three sections.

Third-Party Code

Do not merely write:

```
Prisma  
Vitest
```

Write:

```
Prisma – used to define the SQLite schema, generate migrations and provide  
type-safe database access.
```

```
Vitest – used to run deterministic automated tests through a single npm  
command.
```

Database Design

Explain:

- The Task table.
- Every important column.
- Status restrictions.
- How archiving works.
- That overdue is calculated rather than stored.
- Why there is only one table, assuming Topic remains plain text.

Running It

Include:

- Exact Node.js version.
- Installation command.
- Database setup or migration command.
- Development command.
- Test command.

For example:

```
Node.js: 24.x

npm install
npx prisma migrate deploy
npm run dev
npm test
```

You must eventually verify those instructions from a completely clean clone. Do not blindly copy these commands into the README before testing your actual repository.

9. Commit history matters far more than usual

You need at least six meaningful commits, and the work should be spread over more than one session.

A good sequence:

1. Set up Next.js project with TypeScript
2. Add SQLite task schema and migration
3. Implement task creation and listing
4. Add task editing
5. Add archiving and archived task view
6. Add sorting and overdue indication
7. Add automated task behaviour tests
8. Document installation, database design and dependencies

Each commit should leave the project in a reasonably working state.

Bad commit messages:

```
fix
stuff
update
```

```
final final
final final 2
```

The commit-history section alone is worth **18 marks**, so dumping the whole project into GitHub on submission morning would be impressively self-destructive. lab_one

10. AI transcript requirement

You must submit transcripts covering:

- Planning.
- Code generation.
- Debugging.
- Your instructions and constraints.
- At least one occasion where you reject or correct an unsuitable response.

For example:

Student:

Do not implement deletion. The brief requires tasks to be archived.

AI:

Here is a deleteTask function...

Student:

That violates the brief. Replace it with archivedAt and preserve the row.

The correction must actually correspond to the final code. Do not stage a fake argument with the chatbot afterwards; markers are not goldfish.

11. The marking traps

The biggest traps are:

1. README does not work from a clean clone.

This risks losing the entire 28-mark walkthrough after the allowed retry.

2. Tasks only exist in React state.

They disappear after restart, meaning persistence fails.

3. Archive secretly deletes tasks.

Archived tasks must remain viewable.

4. Overdue is stored as a status.

Only three statuses are permitted.

5. Overdue is stored in the database.

It should be calculated from due date and status.

6. Tests use your normal database.

They must use a throwaway database.

7. One enormous commit.

The rubric explicitly penalises bulk project dumps.

8. Documentation describes a different schema from the code.

12. Your six-day plan

The due date is **4 August 2026**.

lab_one

A realistic plan:

29 July:

Set up repository, Next.js and database schema.

30 July:

Create and list tasks.

31 July:

Edit and archive tasks.

1 August:

Archived view, sorting and overdue display.

2 August:

Tests and bug fixes.

3 August:

README, database documentation and clean-clone test.

4 August:

Final walkthrough, submit repository and AI transcripts.

Your first target should not be "make it beautiful". It should be:

Create one task → store it in SQLite → restart app → task still exists.

Once that works, the rest is controlled expansion rather than interpretive dance.

Path 1: Take me through creating the Next.js repository and SQLite database step by step.

  ...  Sources



How do the files look from a glance, also why is there a
claude.md file?

Worked for 1m 6s >

Ari, nothing looks broken. The folder is named:

```
prisma/migrations
```

—not migrations . It is only created **after the migration command completes successfully**. prisma init creates prisma/schema.prisma and prisma.config.ts , but not the migration history.  Prisma +1

Run these from the project root:

```
npx prisma format
npx prisma validate
PowerShell
```

If validation succeeds, run:

PowerShell

You should then see something like:

```
prisma/
├─ migrations/
│   ├─ migration_lock.toml
│   └─ 20260730..._create_task_table/
│       └─ migration.sql
└─ schema.prisma
```

Then run:

PowerShell

Prisma 7 no longer automatically runs `prisma generate` after `migrate dev` , so this command must be run separately.  Prisma +1

Afterwards, check:

PowerShell

For now, do **not** use `git add .`, because that would also stage these unrelated folders:

```
.agents/  
.claude/  
.windsurf/  
skills-lock.json
```

Once the migration exists, stage only the database work:

```
git add .gitignore package.json package-lock.json prisma.config.ts prisma  
git commit -m "Add SQLite task schema and initial migration"  
git push
```

Paste the full output of:

PowerShell

if it produces an error.

    Sources



is this all ok?

Worked for 25s

Yes, Ari — **the migration has worked.**

What I can confirm from the screenshot:

- `prisma/migrations/..._create_task_table/migration.sql` exists.
- The `Task` table contains the required fields.
- `archivedAt` is nullable, so active tasks can use `null`.
- There is no stored `overdue` field, which is correct.
- Indexes are being created for sorting/filtering.
- `src/generated/prisma` exists, so Prisma Client was generated.
- `dev.db` and `.env` are not appearing in `git status`, meaning they are being ignored correctly.

Your migration SQL looks sensible:

```
CREATE TABLE "Task" (  
  "id" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,  
  "title" TEXT NOT NULL,  
  "description" TEXT NOT NULL,  
  "dueDate" DATETIME NOT NULL,  
  "topic" TEXT NOT NULL,  
  "status" TEXT NOT NULL DEFAULT 'TODO',  
  "archivedAt" DATETIME,  
  "createdAt" DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  "updatedAt" DATETIME NOT NULL  
);
```

`updatedAt` having no SQL default is normal—Prisma supplies it whenever it creates or updates a record.

One important fix before committing

Because `.env` is ignored, somebody cloning the repository will not automatically have:

```
.env
```

Create a file in the project root named:

```
.env.example
```

Put this inside:

env

Later, your README will tell the marker to run:

PowerShell

That allows the project to work from a clean clone.

Ignore the unrelated AI folders

Unless you intentionally created and need them, add these to the bottom of `.gitignore` :

```
# Local coding-assistant configuration
.agents/
.claude/
.windsurf/
skills-lock.json
```

Your tracked `AGENTS.md` and `CLAUDE.md` can remain. The hidden directories are separate local configuration files and are not needed by the todo application.

Then stage the database work

Run:

PowerShell

Check exactly what will be committed:

PowerShell

You should see the intended files under:

```
Changes to be committed
```

Then commit and push:

PowerShell

Do **not** stage `dev.db` or `.env`. The database must be recreated from the committed migration rather than shipping your personal data.



```
PS C:\Users\USER-PC\Documents\coms3011a-lab1> git
add .gitignore .env.example package.json package-
lock.json prisma.config.ts prisma
warning: in the working copy of '.gitignore', LF will be
replaced by CRLF the next time Git touches it
warning: in the working copy of 'package-lock.json', LF will
be replaced by CRLF the next time Git touches it
warning: in the working copy of 'package.json', LF will be
replaced by CRLF the next time Git touches it
The following paths are ignored by one of your .gitignore
files:
```

Show more ▾

Ari, the **LF/CRLF warnings** are harmless. Windows is converting Unix-style line endings to Windows-style line endings; Git will still commit the files normally.

The only actual issue is:

```
.env.example is ignored
```

Your `.gitignore` probably contains:

```
gitignore
```

That pattern ignores both `.env` and `.env.example`.

Fix `.gitignore`

Find the environment-file section and make it:


```
# Environment files
.env*
! env.example
...
```

The exception must appear **after** `.env*`.

Save the file, then run:

```
git add .gitignore .env.example
git status

PowerShell
```

You should see `.env.example` under **Changes to be committed**, while `.env` should remain absent.

If it is still ignored, run:

PowerShell

```
git check-ignore -v .env.example
```

That will show the exact ignore rule causing it.

Once `git status` looks correct:

PowerShell

```
git commit -m "Add SQLite task schema and initial migration"
git push
```

The other files were staged successfully despite the warnings; Git was merely being theatrically informative.



ChatGPT can make mistakes. Check important info.